NS-3 Cheat Sheet

Complete Practical Guide for Networks Lab Experiments (NS-3.42)

1. Essential Headers & Namespace

Every NS-3 script begins by including modular subsystem headers and using the `ns3` namespace.

```
#include "ns3/core-module.h"          // Time, Simulator, CommandLine
#include "ns3/network-module.h"       // NodeContainer, Packet, NetDevice
#include "ns3/internet-module.h"      // IP, TCP, UDP, Routing
#include "ns3/point-to-point-module.h" // P2P links and drop-tail queues
#include "ns3/applications-module.h" // UdpClient, BulkSend, PacketSink
#include "ns3/flow-monitor-module.h" // FlowClassifier, FlowStats
#include "ns3/traffic-control-module.h" // QueueDiscs (FQ-CoDel, FIFO)
#include "ns3/wifi-module.h"          // 802.11 PHY and MAC helpers
#include "ns3/mobility-module.h"      // Position and velocity allocators
#include "ns3/lr-wpan-module.h"       // 802.15.4 Zigbee PHY & MAC
#include "ns3/sixlowpan-module.h"     // IPv6 encapsulation over 802.15.4
#include "ns3/internet-apps-module.h" // Ping and dynamic RIP helpers

using namespace ns3;
```

2. Creating Nodes

Nodes in NS-3 represent empty hardware hosts or routers before protocols and devices are attached.

```
// Create containers to hold end-hosts and routers
NodeContainer hosts;
hosts.Create(4); // Indices: 0, 1, 2, 3

NodeContainer routers;
routers.Create(2); // Intermediate routers r1 (0), r2 (1)

// Accessing individual node pointers
Ptr<Node> n0 = hosts.Get(0);
uint32_t totalHosts = hosts.GetN(); // returns 4
```

3. Configuring Point-to-Point Links

A `PointToPointHelper` configures physical transmission rate, propagation delay, and device buffers[cite: 8].

```
PointToPointHelper p2p;
p2p.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
p2p.SetChannelAttribute("Delay", StringValue("2ms"));

// Set device buffer capacity (e.g., 50 packets)
p2p.SetQueue("ns3::DropTailQueue<Packet>",
             "MaxSize", StringValue("50p"));

// Install creates NetDevices on both nodes connected by a channel
NetDeviceContainer dev0r1 = p2p.Install(hosts.Get(0), routers.Get(0));
```

Queue-Disc Gotcha (Crucial for Packet Drops)

NS-3 automatically attaches a 1000-packet FQ-CoDel queue-disc above device queues[cite: 13, 17]. If you do not remove it, the 50-packet DropTail queue will never fill or drop packets during bottleneck congestion:

```
TrafficControlHelper tch;
// Completely removes default root queue-disc to expose DropTail:
tch.Uninstall(dev0r1);

// Alternative: explicitly install a FifoQueueDisc matching queue size
// tch.SetRootQueueDisc("ns3::FifoQueueDisc", "MaxSize",
//     StringValue("50p"));
// tch.Install(dev0r1);
```

4. Internet Stack & IP Addressing

Installing Internet Stack

Attaches IPv4, IPv6, ARP, UDP, and TCP layer protocols onto nodes[cite: 8].

```
InternetStackHelper internet;
internet.Install(hosts);
internet.Install(routers);
```

Subnetting Point-to-Point Links

Each point-to-point link represents an isolated broadcast domain and must have its own subnet prefix[cite: 13, 17].

```
Ipv4AddressHelper ipv4;

// Subnet for Access Link 1 (Node 0 to Router 1)
ipv4.SetBase("10.1.1.0", "255.255.255.0");
Ipv4InterfaceContainer iface0r1 = ipv4.Assign(dev0r1);

// Subnet for Access Link 2 (Node 1 to Router 1)
ipv4.SetBase("10.1.2.0", "255.255.255.0");
Ipv4InterfaceContainer iface1r1 = ipv4.Assign(dev1r1);

// Subnet for Central Bottleneck Link (Router 1 to Router 2)
ipv4.SetBase("10.1.3.0", "255.255.255.0");
Ipv4InterfaceContainer ifaceBtl = ipv4.Assign(devBtl);

// Subnet for Egress Link (Router 2 to Node 2)
ipv4.SetBase("10.1.4.0", "255.255.255.0");
Ipv4InterfaceContainer ifaceR2N2 = ipv4.Assign(devR2N2);

// Subnet for Egress Link (Router 2 to Node 3)
ipv4.SetBase("10.1.5.0", "255.255.255.0");
Ipv4InterfaceContainer ifaceR2N3 = ipv4.Assign(devR2N3);
```

Target Addressing Rule

In `p2p.Install(nodeA, nodeB)`, index `0` is nodeA and index `1` is nodeB[cite: 13, 17]. To target nodeB from nodeA:

```
// Receiver address is index 1 of the destination link interface
Ipv4Address destAddress = ifaceR2N3.GetAddress(1);
```

Populating Multi-Hop Routing Tables

Computes static shortest paths across all routers and nodes in the topology[cite: 13, 17]:

```
Ipv4GlobalRoutingHelper::PopulateRoutingTables();
```

5. TCP Congestion Control Setup

Set congestion control variants and socket parameters globally before installing any applications or sockets[cite: 14, 17].

```
// Options: "ns3::TcpNewReno", "ns3::TcpCubic", "ns3::TcpBbr", "ns3::TcpVegas"
Config::SetDefault("ns3::TcpL4Protocol::SocketType",
                   StringValue("ns3::TcpCubic"));

// Segment size (MSS): 1024 bytes (Assign 8/9 spec)
Config::SetDefault("ns3::TcpSocket::SegmentSize", UintegerValue(1024));

// Initial Congestion Window in segments
Config::SetDefault("ns3::TcpSocket::InitialCwnd", UintegerValue(10));

// Socket buffer sizes (1 MB = 1048576 bytes)
Config::SetDefault("ns3::TcpSocket::SndBufSize", UintegerValue(1 << 20));
Config::SetDefault("ns3::TcpSocket::RcvBufSize", UintegerValue(1 << 20));

// TCP BBR requires rate pacing to accurately estimate BtlBw:
Config::SetDefault("ns3::TcpSocketState::EnablePacing",
                   BooleanValue(true));
```

6. Applications & Traffic Generators

Port-Based UDP Delivery (Demultiplexing)

Used when multiple applications run across the exact same IP endpoints[cite: 18, 19]. Different destination port numbers ensure the OS demultiplexes packets to the correct application layer server[cite: 19].

6.1.1 Flow 1: Port 5000

```
// Receiver Server on Node 2, Port 5000
UdpServerHelper server1(5000);
ApplicationContainer sApp1 = server1.Install(hosts.Get(2));
sApp1.Start(Seconds(0.0));
sApp1.Stop(Seconds(20.0));

// Sender Client on Node 1 targeting Port 5000
// Rate formula: Interval = (PacketSize * 8) / DesiredDataRate
// For 1 Mbps and 512B packet: (512 * 8) / 1,000,000 = 0.004096s
UdpClientHelper client1(ifaceR2N2.GetAddress(1), 5000);
client1.SetAttribute("MaxPackets", UintegerValue(0xFFFFFFFF)); //
      infinite
client1.SetAttribute("PacketSize", UintegerValue(512));
client1.SetAttribute("Interval", TimeValue(Seconds(512 * 8.0 / 1
     e6)));

ApplicationContainer cApp1 = client1.Install(hosts.Get(1));
cApp1.Start(Seconds(1.0));
cApp1.Stop(Seconds(20.0));
```

6.1.2 Flow 2: Port 6000 (Different Traffic Rate)

```
// Receiver Server on Node 2, Port 6000
UdpServerHelper server2(6000);
ApplicationContainer sApp2 = server2.Install(hosts.Get(2));
sApp2.Start(Seconds(0.0));
sApp2.Stop(Seconds(20.0));

// Sender Client on Node 1 targeting Port 6000 at 2 Mbps
UdpClientHelper client2(ifaceR2N2.GetAddress(1), 6000);
client2.SetAttribute("MaxPackets", UintegerValue(0xFFFFFFFF));
client2.SetAttribute("PacketSize", UintegerValue(512));
client2.SetAttribute("Interval", TimeValue(Seconds(512 * 8.0 / 2
     e6)));

ApplicationContainer cApp2 = client2.Install(hosts.Get(1));
cApp2.Start(Seconds(1.0));
cApp2.Stop(Seconds(20.0));
```

CBR Traffic Generation (OnOffHelper)

Alternative for generating Constant Bitrate UDP traffic without manual interval calculation:

```
// Universal PacketSink listening on Port 9090
PacketSinkHelper udpSink("ns3::UdpSocketFactory",
  InetSocketAddress(Ipv4Address::GetAny(), 9090));
ApplicationContainer sinkApp = udpSink.Install(hosts.Get(2));
sinkApp.Start(Seconds(0.0));
sinkApp.Stop(Seconds(20.0));

// OnOff source generating continuous CBR stream
OnOffHelper onoff("ns3::UdpSocketFactory",
  InetSocketAddress(ifaceR2N2.GetAddress(1), 9090));
onoff.SetConstantRate(DataRate("1Mbps"), 512); // Rate string &
     pkt size
ApplicationContainer srcApp = onoff.Install(hosts.Get(1));
srcApp.Start(Seconds(1.0));
srcApp.Stop(Seconds(20.0));
```

TCP BulkSend Application

Generates a greedy TCP connection that transmits as fast as congestion control allows:

```
// TCP PacketSink listening on Port 8080
PacketSinkHelper tcpSink("ns3::TcpSocketFactory",
  InetSocketAddress(Ipv4Address::GetAny(), 8080));
ApplicationContainer tSinkApp = tcpSink.Install(hosts.Get(3));
tSinkApp.Start(Seconds(0.0));
tSinkApp.Stop(Seconds(20.0));

// BulkSend client connecting to destination Node 3
BulkSendHelper tcpSource("ns3::TcpSocketFactory",
  InetSocketAddress(ifaceR2N3.GetAddress(1), 8080));
tcpSource.SetAttribute("SendSize", UintegerValue(1024));
tcpSource.SetAttribute("MaxBytes", UintegerValue(0)); // 0 =
     unlimited stream

ApplicationContainer tSrcApp = tcpSource.Install(hosts.Get(0));
tSrcApp.Start(Seconds(1.0));
tSrcApp.Stop(Seconds(20.0));
```

7. Simulating Link Failures

Simulate router link drops and link restorations to observe route convergence and packet recovery:

```
// Interface 1 connects Router 1 to the failed path
Ptr<Ipv4> ipR1 = routers.Get(0)->GetObject<Ipv4>();
uint32_t targetIfIndex = 1;

// Schedule link down at 10.0s and recovery at 15.0s
Simulator::Schedule(Seconds(10.0), &Ipv4::SetDown, ipR1,
     targetIfIndex);
Simulator::Schedule(Seconds(15.0), &Ipv4::SetUp,   ipR1,
     targetIfIndex);

// Save routing table snapshots to file before and during failure
Ptr<OutputStreamWrapper> rtStream =
  Create<OutputStreamWrapper>("routes.log", std::ios::out);
Ipv4RoutingHelper::PrintRoutingTableAllAt(Seconds(5.0), rtStream)
     ;
Ipv4RoutingHelper::PrintRoutingTableAllAt(Seconds(12.0), rtStream
     );
```

8. Wireless Networks (Wi-Fi & Zigbee)

Wi-Fi Infrastructure Mode (802.11n)

Connects wireless stations (STAs) to an Access Point (AP) using a shared radio channel[cite: 13, 17].

```
// 1. Channel and PHY layer helpers
YansWifiChannelHelper wifiChannel = YansWifiChannelHelper::
     Default();
YansWifiPhyHelper wifiPhy;
wifiPhy.SetChannel(wifiChannel.Create());
wifiPhy.Set("TxPowerStart", DoubleValue(16.0)); // 16 dBm
     transmission
wifiPhy.Set("TxPowerEnd", DoubleValue(16.0));

// 2. Wi-Fi MAC standard and rate manager
WifiHelper wifi;
wifi.SetStandard(WIFI_STANDARD_80211n);
wifi.SetRemoteStationManager("ns3::ConstantRateWifiManager",
  "DataMode", StringValue("HtMcs7"),
  "ControlMode", StringValue("HtMcs0"));

// 3. Configure SSID and install AP NetDevice
WifiMacHelper wifiMac;
Ssid ssid = Ssid("campus-wifi");

wifiMac.SetType("ns3::ApWifiMac", "Ssid", SsidValue(ssid));
NetDeviceContainer apDevice = wifi.Install(wifiPhy, wifiMac,
     apNode);

// 4. Install Station NetDevices on mobile nodes
wifiMac.SetType("ns3::StaWifiMac", "Ssid", SsidValue(ssid));
NetDeviceContainer staDevices = wifi.Install(wifiPhy, wifiMac,
     staNodes);

// For Ad-Hoc Wi-Fi (VANETs/AODV without APs):
// wifiMac.SetType("ns3::AdhocWifiMac");
// NetDeviceContainer adhocDevs = wifi.Install(wifiPhy, wifiMac,
     adhocNodes);
```

Zigbee (IEEE 802.15.4 LR-WPAN / 6LoWPAN)

Low-rate, short-range wireless communication using IPv6 over 802.15.4[cite: 13, 17]:

```
// 1. Install 802.15.4 NetDevices
LrWpanHelper lrWpan;
NetDeviceContainer lrDevices = lrWpan.Install(zigbeeNodes);
lrWpan.AssociateToPan(lrDevices, 0); // PAN Coordinator ID 0

// 2. Install 6LoWPAN adaptation layer over 802.15.4
SixLowPanHelper sixLowPan;
```

```
NetDeviceContainer sixDevices = sixLowPan.Install(lrDevices);

// 3. Install IPv6 internet stack (disable IPv4)
InternetStackHelper ipStack6;
ipStack6.SetIpv4StackInstall(false);
ipStack6.Install(zigbeeNodes);

// 4. Assign 64-bit IPv6 subnet addresses
Ipv6AddressHelper ipv6;
ipv6.SetBase(Ipv6Address("2001:1::"), Ipv6Prefix(64));
Ipv6InterfaceContainer sixIfaces = ipv6.Assign(sixDevices);

// NOTE: Keep payload sizes <= 80 bytes to prevent fragmentation
```

9. Mobility Models

Wireless nodes require coordinate allocators and mobility models to compute distance, propagation loss, and received signal strength.

Static Position (Access Point / Server)

Fixes a node to an unchanging physical coordinate (x, y, z)[cite: 10, 14, 17]:

```
MobilityHelper mobility;
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");
mobility.Install(apNode);
```

Random Walk 2D (Campus Mobile Nodes)

Nodes choose a random direction and move within a bounding rectangle[cite: 10, 14, 17]. Speed parameter sweeps evaluate performance at 1, 5, and 10 m/s[cite: 7, 14, 16, 17].

```
// Allocate initial positions in a 2D grid
mobility.SetPositionAllocator("ns3::GridPositionAllocator",
  "MinX", DoubleValue(0.0), "MinY", DoubleValue(0.0),
  "DeltaX", DoubleValue(5.0), "DeltaY", DoubleValue(5.0),
  "GridWidth", UintegerValue(5), "LayoutType", StringValue("
     RowFirst"));

// Set movement bounds and speed
mobility.SetMobilityModel("ns3::RandomWalk2dMobilityModel",
  "Bounds", RectangleValue(Rectangle(-100, 100, -100, 100)),
  "Speed", StringValue("ns3::ConstantRandomVariable[Constant=5.0]
     "));
mobility.Install(mobileNodes);
```

Constant Velocity (Vehicular Highway Nodes)

Nodes move continuously in a single direction with a fixed velocity vector[cite: 15, 17]:

```
mobility.SetMobilityModel("ns3::ConstantVelocityMobilityModel");
mobility.Install(vehicleNodes);

// Assign 15 m/s velocity along the X-axis to Vehicle 0:
vehicleNodes.Get(0)->GetObject<ConstantVelocityMobilityModel>()
  ->SetVelocity(Vector(15.0, 0.0, 0.0));
```

10. Packet Tracing & Signal Sniffing

Packet Captures for Wireshark

Generates standard `.pcap` files readable by Wireshark to inspect packet flow and port-based demuxing[cite: 15, 16, 17, 19]:

```
// Trace all point-to-point links
p2p.EnablePcapAll("sim-p2p");

// Trace AP wireless interface
wifiPhy.EnablePcap("sim-wifi-ap", apDevice.Get(0));

// ASCII trace recording enqueue (+), dequeue (-), and drop (d)
     events
AsciiTraceHelper ascii;
p2p.EnableAsciiAll(ascii.CreateFileStream("packet-trace.tr"));
```

Extracting Received Signal Strength (dBm)

Trace callback that reads received signal power directly from the physical layer of Wi-Fi stations[cite: 15, 17]:

```
static void SniffRxSignal(Ptr<const Packet> p, uint16_t
     channelFreq,
                          WifiTxVector txVector, MpduInfo aMpdu,
                          SignalNoiseDbm sn, uint16_t staId) {
  // Prints timestamp and received signal strength in dBm:
  std::cout << Simulator::Now().GetSeconds() << "s | Rx Power: "
            << sn.signal << " dBm\n";
}

// Connect callback to all Wi-Fi PHY receiver layers:
Config::ConnectWithoutContext(
  "/NodeList/*/DeviceList/*/$ns3::WifiNetDevice/Phy/
     MonitorSnifferRx",
  MakeCallback(&SniffRxSignal));
```

11. FlowMonitor Instrumentation & Data Analysis

Collects per-flow metrics across the entire simulation topology[cite: 11, 15, 17].

Installing FlowMonitor

```
FlowMonitorHelper fmh;
Ptr<FlowMonitor> mon = fmh.InstallAll();

// Run simulation until Stop time
Simulator::Stop(Seconds(20.0));
Simulator::Run();

// Ensure in-flight packets are accounted for
mon->CheckForLostPackets();
```

Iterating Over Flow Statistics

```
Ptr<Ipv4FlowClassifier> cl =
  DynamicCast<Ipv4FlowClassifier>(fmh.GetClassifier());
std::map<FlowId, FlowMonitor::FlowStats> stats = mon->
     GetFlowStats();

for (const auto &kv : stats) {
  Ipv4FlowClassifier::FiveTuple t = cl->FindFlow(kv.first);
  const auto &s = kv.second;

  // Filter out reverse TCP ACK flows if not needed
  if (t.destinationPort != 8080 && t.destinationPort != 9090)
     continue;

  double dur = (s.timeLastRxPacket - s.timeFirstTxPacket).
     GetSeconds();
  double mbps = (dur > 0) ? (s.rxBytes * 8.0 / dur / 1e6) : 0;
  double delay = (s.rxPackets > 0) ?
    (s.delaySum.GetSeconds() / s.rxPackets * 1000.0) : 0;
  double loss = (s.txPackets > 0) ?
    (100.0 * (s.txPackets - s.rxPackets) / s.txPackets) : 0;
  double pdr = (s.txPackets > 0) ?
    (100.0 * s.rxPackets / s.txPackets) : 0;

  std::cout << "Flow " << kv.first << " (" << t.sourceAddress
            << " -> " << t.destinationAddress
            << " Port " << t.destinationPort << "):\n"
            << "  Throughput: " << mbps << " Mbps\n"
            << "  Avg Delay:  " << delay << " ms\n"
            << "  Loss:       " << loss << " %\n"
            << "  PDR:        " << pdr << " %\n";
}

// Cleanup simulation memory
Simulator::Destroy();
```

Exporting FlowMonitor Results to XML

```
// Serializes all flow histograms and stats into XML
mon->SerializeToXmlFile("flow-results.xml", true, true);
```

12. Essential Metrics Formulation

Formulas for calculating network metrics from FlowMonitor stats[cite: 15, 17]:

- **Throughput (Mbps):** $\frac{rxBytes \times 8.0}{(t_{lastRx} - t_{firstTx}) \times 10^6}$[cite: 15, 17]
- **Average End-to-End Delay (ms):** $\frac{delaySum.GetSeconds()}{rxPackets} \times 1000.0$[cite: 15, 17]
- **Packet Delivery Ratio (PDR %):** $\frac{rxPackets}{txPackets} \times 100.0$[cite: 15, 17]
- **Packet Loss (%):** $\frac{txPackets - rxPackets}{txPackets} \times 100.0$[cite: 15, 17]
- **Jain's Fairness Index:** $\frac{(\sum x_i)^2}{n \sum (x_i^2)}$[cite: 15, 17]
- **UDP Interval (s):** $\frac{PacketSizeBytes \times 8.0}{DataRateBps}$[cite: 14, 15, 17]

13. Build & Run (Terminal Commands)

Terminal workflows in NS-3.42 using the CMake-based `./ns3` wrapper[cite: 13, 17]:

```
# 1. Copy script into scratch directory
cp my_script.cc scratch/my_script.cc

# 2. Build the NS-3 project
./ns3 build

# 3. Run default simulation
./ns3 run scratch/my_script

# 4. Run with custom command-line arguments
./ns3 run "scratch/my_script --bw=5Mbps --delay=20ms --q=100"

# 5. Enable module logging during execution
NS_LOG="UdpClient=level_info" ./ns3 run scratch/my_script
```